

Learning Logistic Circuits

Yitao Liang and Guy Van den Broeck

Computer Science Department
University of California, Los Angeles
{yliang, guyvdb}@cs.ucla.edu

Abstract

This paper proposes a new classification model called logistic circuits. On MNIST and Fashion datasets, our learning algorithm outperforms neural networks that have an order of magnitude more parameters. Yet, logistic circuits have a distinct origin in symbolic AI, forming a discriminative counterpart to probabilistic-logical circuits such as ACs, SPNs, and PSDDs. We show that parameter learning for logistic circuits is convex optimization, and that a simple local search algorithm can induce strong model structures from data.

1 Introduction

Circuit representations are a promising synthesis of symbolic and statistical methods in AI. They are “deep” layered data structures with statistical parameters, yet they also capture intricate structural knowledge. Recently, many representations have been proposed for learning tractable probability distributions: arithmetic circuits (?), weighted SDD (?), PSDD (?), cutset networks (?) and sum-product networks (SPNs ?). Collectively, these approaches achieve the state of the art in discrete density estimation and vastly outperform classical probabilistic graphical model learners (?; ?; ?; ?; ?). However, we have not observed the same success when deploying circuit representations for *classification or discriminative learning*. Probabilistic circuit classifiers significantly lag behind the performance of neural networks (?).

In this paper, we propose a new classification model called *logistic circuits*, which shares many syntactic properties with the representations mentioned earlier. One can view logistic circuits as the discriminative counterpart to probabilistic circuits. Owing to their elegant properties, learning the parameters of a logistic circuit can be reduced to a logistic regression problem and is therefore convex. Learning logistic circuit structure is reduced to a simple local search problem using primitives from the probabilistic circuit learning literature (?).

We run experiments on standard image classification benchmarks (MNIST and Fashion) and achieve accuracy higher than much larger MLPs and even CNNs with an order of magnitude more parameters. For example, logistic circuits obtain 99.4% accuracy on MNIST. Compared to other

tractable learners on MNIST, and the state-of-the-art discriminative SPN learner in particular (?), our logistic circuit learner cuts the error rate by a factor of three. Furthermore, we show our learner is highly data efficient, managing to still learn well with limited data.

This paper proceeds as follows. Section 2 introduces the syntax and semantics of logistic circuits. Sections 3 and 4 describe our parameter and structure learning algorithms, which Section 5 evaluates empirically. Section 6 elaborates on the connection with tractable generative models, after which we conclude with related and future work.

2 Representation

This section introduces the logistic circuit representation.

Notation We use uppercase X to denote a Boolean random variable and lowercase x for a specific assignment to it. Interchangeably, we also interpret Boolean random variables as logical variables. A set of variables $\mathbf{X}$ and their joint assignments $\mathbf{x}$ are denoted in bold. A complete assignment $\mathbf{x}$ to all variables is a possible world, or interchangeably, a data sample. Literals are variables X or their negation $\neg X$. Logical sentences are constructed from literals and connectives such as AND and OR in the standard way. An assignment $\mathbf{x}$ that satisfies a logical sentence α is denoted as $\mathbf{x} \models \alpha$.

2.1 Logical Circuits

A logical circuit is a directed acyclic graph representing a logical sentence, as depicted in Figure 1a (ignoring parameters for now). Each inner node is either an AND gate or an OR gate.¹ A leaf (input) node represents a Boolean literal, that is, X or $\neg X$, where the node can only be satisfied if X is set to 1 (true) respectively 0 (false).

The following properties are key for logical circuits to be well-behaved (?). An AND gate is *decomposable* if its inputs depend on disjoint sets of variables. For example, the top-most AND gates in Figure 1a depend on A in their one input and on $\{B, C, D\}$ in their other input. When an AND gate has two inputs, they are called its prime (left) and sub (right). An OR gate is *deterministic* if for any single complete assignment, at most one of its inputs can be set to 1.

¹We consider negation-normal-form circuits where no negation is allowed except at the leafs/inputs (?).

where $\Pr_{\mathbf{q}}(\cdot)$ is the fully-factorized distribution where each variable in $\mathbf{X}$ has the probability assigned by $\mathbf{q}$.

Logistic circuit semantics now support continuous data (after normalizing to $[0, 1]$), simply by replacing Boolean flow with probabilistic flow in Definition 2. Note that probabilistic circuit flow has Boolean circuit flow as a special case, when $\mathbf{q}$ happens to be binary. Furthermore, due to the determinism and decomposability properties, the probabilities in Definition 3 can be computed efficiently, together with all probabilistic circuit flows and weight functions in the logistic circuit. We defer the discussion of these computational details to Section 3.4. In the rest of this paper, we will abuse notation and have $\mathbf{x}$ refer to Boolean inputs as well as continuous inputs $\mathbf{q}$ interchangeably.

3 Parameter Learning

A natural next question is how to learn logistic circuit parameters from complete data, for a fixed given circuit structure (structure learning is discussed in Section 4). Furthermore, we ask whether those learned parameters are guaranteed to be optimal, globally minimizing a loss function. We address these questions by showing how parameter learning can be reduced to logistic regression on a modified set of features, owing to logistic circuits' strong properties.

3.1 Special Cases

Before presenting the general reduction, we briefly discuss two special cases that establish some intuition.

Linear Weight Functions Consider a vanilla logistic regression model on input variables (features) $\mathbf{X}$. Does there exist an equivalent logistic circuit with the same weight function? For sample $\mathbf{x}$, logistic regression with parameters θ would have weight function $\mathbf{x} \cdot \theta$. Following Definition 2, we obtain such a simple weight function (linear in the input variables) by placing OR gates over complementary pairs of literals and associating a θ parameter which each wire (see Figure 2).² A large parent AND gate collects these variable-wise weights into a single linear sum. Finally, an OR gate at the root adds the bias term regardless of the input.

Proposition 1. *For each classical logistic regression model, there exists an equivalent logistic circuit model.*

Boolean Flow Indicators Next, let us consider a special case that makes no assumptions about circuit structure, but that requires the inputs to be fully binary. Such a circuit would have Boolean flows through every wire. Instead of working with the input variables $\mathbf{X}$, we can introduce new features that are indicator variables, telling us how the example propagates through the circuit, and which wires have a Boolean flow that reaches the circuit root. The circuit flows (indicators) decide which parameters are summed into the weight function; this process has been implicitly revealed in Figure 1a. By introducing such indicators, we can always obtain a linear weight function of composite features that

²The negated variable inputs and parameters $\theta_{\neg X}$ are redundant, but we keep them for the sake of consistency. Alternatively, we can fix $\theta_{\neg X} = 0$ for all X to remove this redundancy.

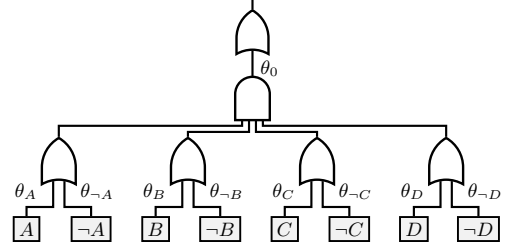


Figure 2: Logistic regression represented as a logistic circuit

are extracted from sample $\mathbf{x}$. Next, we generalize this idea of introducing wire features to arbitrary logistic circuits.

3.2 Reduction to Logistic Regression

We will now consider the most general case, with continuous input data and no assumptions on the circuit structure.

Proposition 2. *Any logistic circuit model can be reduced to a logistic regression model over a particular feature set.*

Corollary 3. *Logistic circuit cross-entropy loss is convex.*

To prove Proposition 2, we need to rewrite the classification distribution in Definition 2 as follows.

$$\Pr(Y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{x} \cdot \theta)}.$$

Here, $\mathbf{x}$ is some vector of features extracted from the raw example $\mathbf{x}$. This feature vector can only depend on $\mathbf{x}$; not on the parameters θ . Thus, the fundamental question is whether we can decompose $g_n(\mathbf{x})$ into $\mathbf{x} \cdot \theta$ for all nodes n . We prove this to be true by induction:

- **Base case:** n is a leaf (input) node. It is obvious g_n can be expressed as $\mathbf{x} \cdot \theta$ since g_n always equals 0.
- **Induction step:** assume g of all the nodes under node n can be expressed as $\mathbf{x} \cdot \theta$. We need to consider two cases:

1. If n is an AND gate having (w.l.o.g.) two children, prime p and sub s . Given $g_p = \mathbf{x}_p \cdot \theta_p$ and $g_s = \mathbf{x}_s \cdot \theta_s$,

$$\begin{aligned} g_n &= \mathbf{x}_p \cdot \theta_p + \mathbf{x}_s \cdot \theta_s \\ &= \begin{bmatrix} \mathbf{x}_p \\ \mathbf{x}_s \end{bmatrix} \cdot \begin{bmatrix} \theta_p \\ \theta_s \end{bmatrix}. \end{aligned}$$

2. If n is an OR gate with (child node, wire parameter) inputs $\{(c_1, \theta_1), \dots, (c_m, \theta_m)\}$. Given $g_{c_i} = \mathbf{x}_{c_i} \cdot \theta_{c_i}$,

$$\begin{aligned} g_n &= \sum_i f(n, \mathbf{x}, c_i) \cdot (\mathbf{x}_{c_i} \cdot \theta_{c_i} + \theta_i) \\ &= \begin{bmatrix} f(n, \mathbf{x}, c_1) \cdot \mathbf{x}_{c_1} \\ f(n, \mathbf{x}, c_1) \\ \vdots \\ f(n, \mathbf{x}, c_m) \cdot \mathbf{x}_{c_m} \\ f(n, \mathbf{x}, c_m) \end{bmatrix} \cdot \begin{bmatrix} \theta_{c_1} \\ \theta_1 \\ \vdots \\ \theta_{c_m} \\ \theta_m \end{bmatrix}. \end{aligned}$$

Note that this proof holds true regardless of whether the input sample $\mathbf{x}$ is binary or real-valued. With this proof, it is obvious that learning the parameters of a logistic circuit

is equivalent to logistic regression on features $\mathbf{x}$. We refer readers to ? (?) for a detailed proof that logistic regression is convex.

Given this correspondence, any convex optimization technique can now be brought to bear on the problem of learning the parameters of a logistic circuit. In particular, we use stochastic gradient descent for this task.

3.3 Global Circuit Flow Features

In the proof of Proposition 2, features $\mathbf{x}$ are computed recursively by induction. However, it is not clear what these features represent, and how they are connected to the input data. In this section we assign semantics to those extracted features. They are the *global circuit flow* of the observed example through the circuit. Global circuit flow is defined with respect to the root of a logistic circuit.

Definition 4 (Global Circuit Flow). *Consider a logistic circuit over variables $\mathbf{X}$ rooted at OR gate r . The global circuit flow $f_r(n, \mathbf{x}, c)$ of input $\mathbf{x}$ between parent n and child c is defined inductively as follows. The global circuit flow between root r and its child c is the (local) probabilistic circuit flow: $f_r(r, \mathbf{x}, c) = f(r, \mathbf{x}, c)$. Then, for any node n with parents $v_1, \dots, v_m$, we have that*

- if n is an AND gate, global flow from child c is

$$f_r(n, \mathbf{x}, c) = \sum_{i=1}^m f_r(v_i, \mathbf{x}, n),$$

- if n is an OR gate, global flow from child c is

$$f_r(n, \mathbf{x}, c) = f(n, \mathbf{x}, c) \cdot \sum_{i=1}^m f_r(v_i, \mathbf{x}, n).$$

The red wires in Figure 1a have a global circuit flow of 1 for the given Boolean input. In general, global circuit flow assigns a continuous probability value to each wire.

Based on global circuit flow, we postulate the following alternative semantics for logistic circuits.

Definition 5 (Logistic Circuit Alternative Semantics). *Let $\mathcal{W}$ be the set of all wires (n, θ, c) between OR gates n and children c with parameters θ . Then, a logistic circuit rooted at node r defines the weight function*

$$g_r(\mathbf{x}) = \sum_{(n, \theta, c) \in \mathcal{W}} f_r(n, \mathbf{x}, c) \cdot \theta.$$

Note that the definition of global circuit flows, as well as our alternative semantics, follow a top-down induction. In contrast, the original semantics in Definition 2 follow a bottom-up induction. We resolve this discrepancy next.

Proposition 4. *The features $\mathbf{x}$ constructed in the proof of Proposition 2 are equivalent to global flows $f_r(n, \mathbf{x}, c)$.*

Corollary 5. *The bottom-up semantics of Definition 2 and the top-down semantics of Definition 5 are equivalent.*

We defer the proof of this proposition to Appendix ??.

Recall that without parameters, a logistic circuit is simply a logical circuit, which means that gates in a logistic circuit have real meaning: they correspond to some logical sentence. Hence, the values of global circuit flow features $\mathbf{x}$ correspond to probabilities of these logical sentences according

to the input vector $\mathbf{x}$. This provides us with a precious opportunity to assign meaning to the features learned by logistic circuits. We will revisit this point in Section 5.4, where we also visualize some global circuit flow features.

3.4 Computing Global Flow Features Efficiently

While logistic circuit parameter learning is convex, we would like to also guarantee that the required feature computation is tractable. This section discusses efficient methods to calculate global flow features $\mathbf{x}$ (i.e., $f_r(n, \mathbf{x}, c)$) from training samples $\mathbf{x}$ offline, before parameter learning.

As is clear from Definition 3, circuit flows make extensive use of node probabilities. We design our computation to consist of two parts, and dedicate the first part to the calculation of node probabilities. The first part is a bottom-up linear pass over the circuit starting with leaf nodes whose probabilities are directly provided by the input sample; see the details in Appendix ??. The second part makes use of these node probabilities to calculate the global circuit flow features in linear time. It is a top-down implementation of the recursion in Definition 4; see its details in Appendix ??. Note that these computations correspond to the partial derivative computations used in arithmetic circuits for the purpose of probabilistic inference (?).

Our algorithm is completely compatible with fast vector arithmetic: instead of inputting one single sample each time, one can directly supply the algorithms with a vector of samples (e.g., a mini batch), and this yields significant speedups.

4 Structure Learning

This section presents an algorithm to learn a compact logical circuit structure for logistic circuits from data. For simplicity of designing the primitive operations, we assume AND gates always have two inputs (prime and sub).

4.1 Learning Primitive

The split operation was first introduced to modify the structure of PSDD circuits (?). We adopt it here with minor changes³ as the primitive operation for our structure learning algorithm. Splitting an AND gate happens by imposing two additional constraints that are *mutually exclusive* and *exhaustive*, in particular by making two opposing variable assignments. Executing a split creates partial copies of the gate and some of its decedents. Furthermore, one can choose to duplicate additional nodes up to a fixed depth (3 in our experiments). We refer readers to ? (?) for further details on the algorithm for executing splits.

Splits are ideal primitives to change the classifier induced by a logistic circuit: they directly affect the circuit flows (see Figure 3). By imposing constraints on AND gates, splits alter the node probabilities associated with the affected AND gates. Following Definition 3, the circuit flows on the wires out of those AND gates adapt accordingly. While Figure 3 focuses on the immediately affected wires, the effect of a split on circuit flows can propagate downward for several levels, depending on the depth of node duplication. Still the

³Compared to the splits in LearnPSDD (?), we do not limit constraints to be on primes.

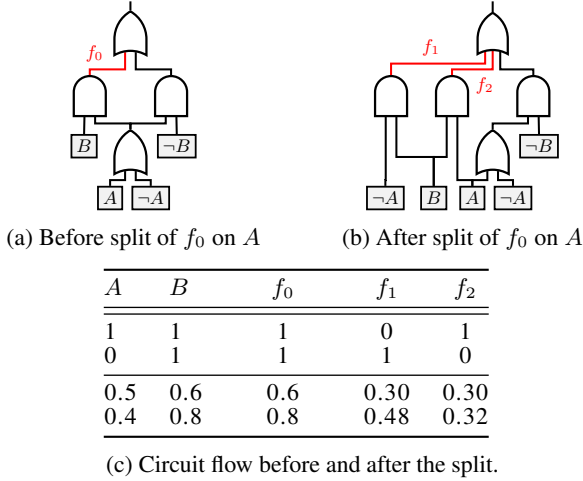


Figure 3: A split changes the circuit flow.

effects of a split on both the structure of a logistic circuit and the circuit flows are very local and contained in the sub-circuit rooted at the OR parent of the split AND gate. However, its effect on the parameters is global. Once a split is executed, the whole parameter set needs to be re-trained.

4.2 Learning Algorithm

The overall structure learning algorithm for logistic circuits, built on top of the split operation, proceeds as follows. Iteratively, one split is executed to change the structure, followed by parameter learning. We only consider single-variable split constraints and first select which AND gate to split, followed by a selection of which variable to split on.

When using gradient descent, one hopes that the parameter on the AND gate output consistently has its partial derivatives pointing in the same direction for all training examples. This will steadily push the parameter to a large magnitude.

If this is not the case, we will use splits to alter the flow of examples through the circuit. Specifically, those AND gates whose associated output parameter has a large variance of its partial derivative (that is, the derivative of the loss function w.r.t. that parameter) requires splitting for the parameters to improve. We simply select the AND gate whose output parameter has the highest training variance.

Given an AND gate to split, we consider candidate variables X to execute the split with. We construct two sets of training examples that affect this node: in one group, each example is weighted by the marginal probability of X ; in the other, with the marginal probability of $\neg X$. Next, we calculate the within-group weighted variances of the partial derivatives. The variable with the smallest weighted variances gets picked, as this suggests the split will introduce new parameters with gradients that align in one direction.

5 Empirical Evaluation

In this section, we empirically evaluate the competitiveness of our learner on three aspects: classification accuracy,

Table 1: Classification accuracy of logistic circuits in context with commonly used existing models. We report the details of those existing models in Appendix ??.

ACCURACY % ON DATASET	MNIST	FASHION
BASILINE: LOGISTIC REGRESSION	85.3	79.3
BASILINE: KERNEL LOGISTIC REGRESSION	97.7	88.3
RANDOM FOREST	97.3	81.6
3-LAYER MLP	97.5	84.8
RAT-SPN (?)	98.1	89.5
SVM WITH RBF KERNEL	98.5	87.8
5-LAYER MLP	99.3	89.8
LOGISTIC CIRCUIT (BINARY)	97.4	87.6
LOGISTIC CIRCUIT (REAL-VALUED)	99.4	91.3
CNN WITH 3 CONV LAYERS	99.1	90.7
RESNET (?)	99.5	93.6

Table 2: Number of parameters of logistic circuits in context with existing SGD-based models, when achieving the classification accuracy reported in Table 1

NUMBER OF PARAMETERS	MNIST	FASHION
BASILINE: LOGISTIC REGRESSION	<1K	<1K
BASILINE: KERNEL LOGISTIC REGRESSION	1,521 K	3,930K
LOGISTIC CIRCUIT (REAL-VALUED)	182K	467K
LOGISTIC CIRCUIT (BINARY)	268K	614K
3-LAYER MLP	1,411K	1,411K
RAT-SPN (?)	8,500K	650K
CNN WITH 3 CONV LAYERS	2,196K	2,196K
5-LAYER MLP	2,411K	2,411K
RESNET (?)	4,838K	4,838K

model complexity, and data efficiency.⁴ Moreover, we visualize the most important active feature with regards to the given sample to provide local interpretation for why the learned logistic circuit makes such classification.

5.1 Setup & Data Preprocessing

We choose MNIST and Fashion⁵ as our testbeds. Since logistic circuits are intended for binary classification, we use the standard “one vs. rest” approach to construct an ensemble multi-class classifier such that our method can be evaluated on these two datasets. When running the binary logistic circuit, we transform pixels that are smaller than their mean plus 0.05 standard deviation to 0 and the rest to 1. When running the real-valued version, we transform pixels to $[0, 1]$ by dividing them by 255. All experiments start with a pre-defined initial structure; we defer its details to Appendix ??. The learned structure with the highest F1 score on validation after 48 hours of running is used for evaluation. All experiments are run on single CPUs.

⁴Open-source code and experiments are available at <https://github.com/UCLA-StarAI/LogisticCircuit>.

⁵A dataset of Zalando’s images, intended as a more challenging drop-in replacement of MNIST (?).

Table 3: Comparison of logistic circuits with MLPs when trained with different percentages of the dataset.

ACCURACY % WITH % OF TRAINING DATA	MNIST			FASHION		
	100%	10%	2%	100%	10%	2%
5-LAYER MLP	99.3	98.2	94.3	89.8	86.5	80.9
CNN WITH 3 CONV LAYERS	99.1	98.1	95.3	90.7	87.6	83.8
LOGISTIC CIRCUIT (BINARY)	97.4	96.9	94.1	87.6	86.7	83.2
LOGISTIC CIRCUIT (REAL-VALUED)	99.4	97.8	96.1	91.3	87.8	86.0

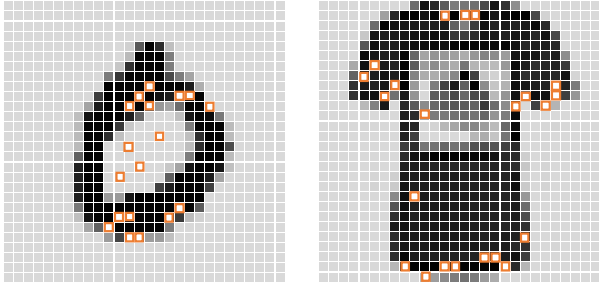


Figure 4: Visualization of the single compositional feature that contributes most to the classification probability with regards to the input image. Features are marked in orange. Left: a digit 0 from MNIST. Right: a t-shirt from Fashion.

5.2 Classification Accuracy

Table 1 summarizes the classification accuracy on test data. Learning a logistic circuit on the binary data is on par with a 3-layer MLP; the real-valued version outperforms 5-layer MLPs and even CNNs with 3 convolutional layers. The fact that logistic circuits achieve better accuracy than CNNs is surprising, since logistic circuits do not use convolutions, which are specifically designed to exploit image invariances.

In addition, we would like to emphasize our comparison with two of the baselines. As parameter learning of logistic circuits is equivalent to logistic regression, one can view structure learning of logistic circuits as a process of constructing composite features from raw samples. The significant improvement over standard logistic regression demonstrates the effectiveness of our method in extracting valuable features; using kernel logistic regression can only partially bridge the gap in performance, yet as shown later, it does so at the cost of introducing many more parameters.

We also want to call attention to our comparison with RAT-SPN, the current state of the art in discriminative learning for probabilistic circuits. SPN is another form of circuit representation, with less restrictive structure. Parameter learning in SPN is not convex and generally requires other techniques such as EM or non-convex optimization. The empirical observation that our method achieves significantly better classification accuracy than RAT-SPN demonstrates that in structure learning, imposing more restrictions on the model’s structural syntax may be beneficial. The syntactic restriction of logistic circuits requires decomposability and determinism; without them, convex parameter learning does not appear to be possible. As structure learning is built on

top of parameter learning, a well-behaved parameter learning loss with a unique optimum can provide more informative guidance about how to adapt the structure, leading to a more competitive structure learning algorithm overall.

5.3 Model Complexity & Data Efficiency

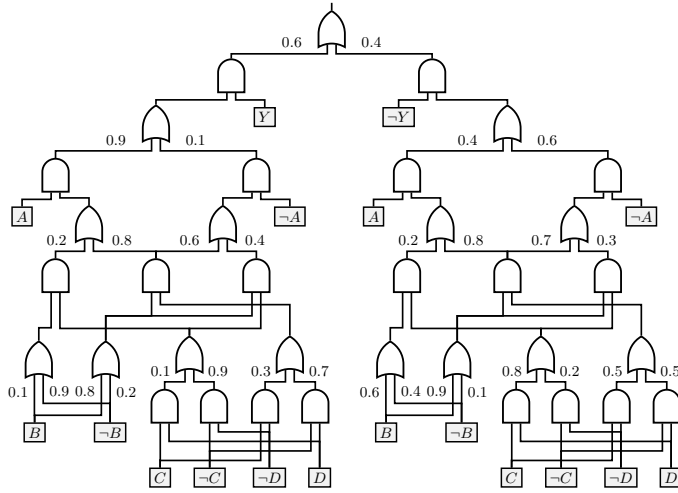
Table 2 summarizes the size of all compared models when achieving the reported accuracy. We can conclude that logistic circuits are significantly smaller than the alternatives, despite attaining higher accuracy.

We design the next set of experiments to specifically investigate how well our structure learning algorithm performs under the setting where the number of training samples is limited. We have two additional sets of experiments, where only 2% and 10% of the original training data is supplied. Table 3 summarizes the performance in this limited-data setting. We mainly compare against a 5-layer MLP and CNN with 3 convolutional layers, whose performance is on par with our method under the full-data setting. As summarized in Table 3, except on MNIST with 10% training samples, real-valued logistic circuits achieve the best classification accuracy. Moreover, in both versions of logistic circuits, when the available training samples are reduced from 100% to 2%, the accuracy only drops by around 3% when evaluating on MNIST; around 5% on Fashion. In contrast, a much larger drop occurs for 5-layer MLP and CNN. Specifically, MLP’s accuracy drops by 5% (9%) while CNN’s accuracy drops by 4% (7%) on MNIST (Fashion). This small magnitude of accuracy decrease illustrates how data efficient our proposed structure learning algorithm is.

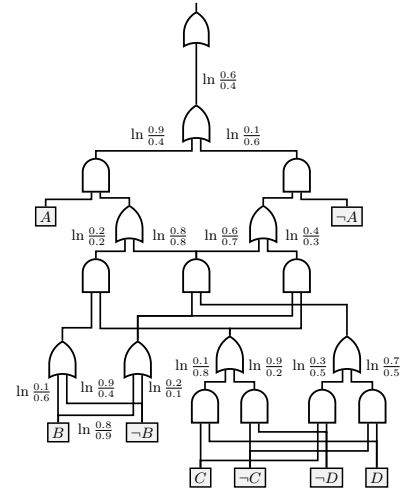
Except on MNIST with 10% training samples, real-valued logistic circuits achieve the best classification accuracy. From a top-down perspective, each OR gate of a logistic circuit presents a weighted choice between its wires. Hence, one can view a logistic circuit as a decision diagram. Under this perspective, splits refine OR gates’ branching rules. As each branching rule naturally applies to multiple samples, we hypothesize that the splits selected by our structure learning algorithm reflect the general conditional feature information present in the dataset.

5.4 Local Explanation

Next, we aim to share some insights about how to explain the learned logistic circuit. Specifically, we investigate the question: “Why does the logistic circuit classify a given sample x as y ?” Since any logistic circuit can be reduced to a logistic regression classifier, we can easily find the active global flow



(a) Probabilistic circuit for joint distribution $\Pr(Y, A, B, C, D)$



(b) Logistic circuit for $\Pr(Y = 1 | A, B, C, D)$

Figure 5: A probabilistic circuit with parallel structures under class variable Y and its equivalent logistic circuit for predicting Y

feature that contributes most to the given sample's classification probability. That is, the feature that maximizes $\mathbf{x} \cdot \theta$. We visualize one such feature for MNIST data and one for Fashion in Figure 4 by marking the variables used in the their corresponding logical sentences.

6 Connection to Probabilistic Circuits

In recent years, a large number of tractable probabilistic models have been proposed as a target representation for generative learning of a joint probability distribution: arithmetic circuits (?), weighted SDD (?), PSDD (?), cutset networks (?) and sum-product networks (SPNs) (?). These representations have various syntactic properties. Some put probabilities on terminals, others on edges. Some use logical notation (AND, OR), others use arithmetic notation ($\times, +$). Nevertheless, they are all circuit languages built around the properties of decomposability and/or determinism.

For our purpose, we consider a simple probabilistic circuit language based on the logistic circuit syntax, where now the θ parameters are assumed to be normalized probabilities.⁶

Definition 6 (Probabilistic Circuit Semantics). *A probabilistic circuit node n defines the following joint distribution.*

- If n is a leaf (input) node, then $\Pr_n(\mathbf{x}) = [\mathbf{x} \models n]$.
- If n is an AND gate with children $c_1, \dots, c_m$, then

$$\Pr_n(\mathbf{x}) = \prod_{i=1}^m \Pr_{c_i}(\mathbf{x}).$$

- If n is an OR gate with (child node, wire parameter) inputs $(c_1, \theta_1), \dots, (c_m, \theta_m)$, then

$$\Pr_n(\mathbf{x}) = \sum_{i=1}^m \Pr_{c_i}(\mathbf{x}) \cdot \theta_i.$$

⁶We also assume *smoothness* (?).

Figure 5a shows a probabilistic circuit for the joint distribution $\Pr(Y, A, B, C, D)$. This tractable circuit language is a relaxation of PSDDs (?) and a specific type of SPN (?) where determinism holds throughout. It is also a type of arithmetic circuit.

We are now ready to connect logistic and probabilistic circuits. It is well known that logistic regression is the discriminative counterpart of a naive Bayes generative model (?). A similar correspondence holds between our logistic and probabilistic circuits.

Proposition 6. *Consider a probabilistic circuit whose structure is of the form $(Y \wedge \alpha) \vee (\neg Y \wedge \beta)$, where sub-circuits α and β are structurally identical. Then, there exists an equivalent logistic circuit for the conditional probability of Y in the probabilistic circuit. Moreover, this logistic circuit has structure $\vee \alpha$ and its parameters can be computed in closed form as log-ratios of probabilistic circuit probabilities.*

We first depict this correspondence intuitively in Figure 5. The logistic circuit has the same structure as the two halves of the probabilistic circuit, and its parameters are computed from the probabilistic circuit probabilities. The distributions $\Pr(Y = 1 | A, B, C, D)$ represented by the circuits in Figures 5a and 5b are identical.

Formal Correspondence Next, we present the formal proof of this correspondence for binary $\mathbf{x}$. Recall that in our circuits, only the input wires of OR gates are parameterized. Let $\mathcal{W}_\delta$ be the set that contains all these wires in circuit δ :

$$\mathcal{W}_\delta = \{(n, c) \mid c \text{ is a gate with parent OR gate } n\}.$$

After expanding the equations in Definition 6 and following the top-down definition of global circuit flow (i.e., following Definition 4), one finds that the joint distribution induced by a probabilistic circuit δ can be rewritten as

$$\Pr_\delta(\mathbf{x}) = \prod_{(n, c) \in \mathcal{W}_\delta} f_\delta(n, \mathbf{x}, c) \cdot \theta_{(n, c)}^\delta.$$

We will exploit this finding in the derivation of the conditional distribution induced by the probabilistic circuit $\gamma = (Y \wedge \alpha) \vee (\neg Y \wedge \beta)$.

$$\begin{aligned} \Pr_\gamma(Y = 1 \mid \mathbf{x}) &= \frac{\Pr_\gamma(Y=1) \Pr_\alpha(\mathbf{x})}{\Pr_\gamma(Y=0) \Pr_\beta(\mathbf{x}) + \Pr_\gamma(Y=1) \Pr_\alpha(\mathbf{x})} \\ &= \frac{1}{1 + \frac{\Pr_\gamma(Y=0) \Pr_\beta(\mathbf{x})}{\Pr_\gamma(Y=1) \Pr_\alpha(\mathbf{x})}} \\ &= \frac{1}{1 + \frac{\Pr_\gamma(Y=0) \prod_{(n,c) \in \mathcal{W}_\beta} f_\beta(n, \mathbf{x}, c) \theta_{(n,c)}^\beta}{\Pr_\gamma(Y=1) \prod_{(n,c) \in \mathcal{W}_\alpha} f_\alpha(n, \mathbf{x}, c) \theta_{(n,c)}^\alpha}} \end{aligned}$$

As stated in Proposition 6 and shown in Figure 5, sub-circuits α and β share the same structure. Therefore, we can further simplify this equation as follows.

$$\begin{aligned} \Pr_\gamma(Y = 1 \mid \mathbf{x}) &= \frac{1}{1 + \frac{\Pr_\gamma(Y=0)}{\Pr_\gamma(Y=1)} \prod_{(n,c) \in \mathcal{W}_\alpha} f_{\vee\alpha}(n, \mathbf{x}, c) \frac{\theta_{(n,c)}^\beta}{\theta_{(n,c)}^\alpha}} \\ &= \frac{1}{1 + \exp[-g(\mathbf{x})]} = \Pr_{\vee\alpha}(Y = 1 \mid \mathbf{x}) \end{aligned}$$

where

$$g(\mathbf{x}) = \log \frac{\Pr_\gamma(Y=1)}{\Pr_\gamma(Y=0)} + \sum_{(n,c) \in \mathcal{W}_\alpha} f_{\vee\alpha}(n, \mathbf{x}, c) \log \frac{\theta_{(n,c)}^\alpha}{\theta_{(n,c)}^\beta} \quad (2)$$

$$= \theta_{root}^{\vee\alpha} + \sum_{(n,c) \in \mathcal{W}_\alpha} f_{\vee\alpha}(n, \mathbf{x}, c) \cdot \theta_{(n,c)}^{\vee\alpha}. \quad (3)$$

The transformation from Equation 2 to 3 expresses the logistic circuit parameters as the log-ratios of probabilistic circuit probabilities. For example, the class priors captured in the output wires of α and β are now combined as a log-ratio to form the bias term for $\vee\alpha$, expressed by the root parameter. This proof also provides us with a new perspective to understand the semantics of the learned parameters. The parameters represent the log-odds ratio of the features given different classes. Note that by Bayes' theorem, a naive Bayes model would derive its induced distribution in a sequence of steps similar to the ones above, resulting in Equation 2. Given this correspondence, one can also view our proposed structure learning method as a way to construct meaningful features for a naive Bayes classifier. We know that after training, naive Bayes classifiers are equivalent to logistic regression classifiers (as in Equation 3).

7 Related Work

? (?) proposed the first parameter learning algorithm for discriminative SPNs, using MPE inference as a sub-routine. Without the support of the determinism property, parameter learning of general SPNs is a relatively harder question than its logistic circuit counterpart, since it is non-convex. ? (?) boost the accuracy of SPNs on MNIST to 97.6% by extracting more representative features from raw inputs based on

the Hilbert-Schmidt independence measure. ? (?) further improved the classification ability of SPNs by drastically simplifying SPN structure requirements and utilizing a loss objective that hybrids cross-entropy (discriminative learning) with log-likelihood (generative learning). ? (?) developed a discriminative structure learning algorithm for arithmetic circuits. The method updates the circuit that represents a corresponding conditional random field (CRF) model by adding features conditioned on arbitrary evidence to the model. This work further relaxes decomposability and smoothness properties of ACs for a more compact representation. However, it targets the setting where there are a large number of output variables, not single-variable classification. All the aforementioned literature conforms to a common trend of abandoning properties of the chosen circuit representations for easier structure learning and better prediction accuracy. They argue that those special syntactic restrictions complicate the learning process. On the contrary, this paper chooses perhaps the most structure-restrictive circuit as the target representation. Instead of relaxing the target representation's syntactical requirements, our proposed method fully leverages the valuable properties that stem from these restrictions, and in particular convexity.

8 Conclusions

We have presented logistic circuits, a novel circuit-based classification model with convex parameter learning and a simple structure learning procedure based on local search. Logistic circuits outperform much larger classifiers and perform well in a limited data regime. Compared to other symbolic, circuit-based approaches, logistic circuits present a leap in performance on image classification benchmarks. Future work includes support for convolution, parameter tying, and structure sharing in the logistic circuits framework. Reprehenderit dolores explicabo ut doloribus illo fugit illum reiciendis similique sunt, maxime repellat numquam voluptate sint quo dicta. Fuga culpa atque nesciunt laborum pariatur eveniet rem id sequi, quaerat obcaecati dignissimos totam fugiat quam recusandae, at cupiditate nesciunt teneatur, fuga nostrum dolorem error ullam illo consectetur. Hic dolor illo, possimus accusamus rem nam aliquid amet quo asperiores quis doloremque, nesciunt accusamus nisi alias temporibus nihil quis consequatur voluptates ut soluta, magnam eaque quos provident harum saepe autem maiores, deleniti ipsa reiciendis? Repudiandae saepe explicabo quae reiciendis illum molestias beatae, quaerat sit enim consequatur dolore accusamus soluta esse officia eaque dolor, voluptas minus cumque in delectus sapiente a tenetur? Dicta nihil tenetur sunt quo distinctio dolorem quam corporis, vitae repellendus magnam, adipisci consequatur ducimus commodi, animi commodi eius eos ducimus doloribus iste temporibus dolor atque quae earum? Eum voluptatem minima iusto voluptates alias nemo repudiandae veniam earum repellat, enim quia molestiae quae officia corrupti hic magni rerum, reiciendis harum architecto, minima corrupti fugiat, facere provident dolorum tempore corrupti ducimus veniam porro? Ipsum reprehenderit quidem fuga, dignissimos unde laborum? Suscipit distinctio saepe voluptatum cumque veritatis velit, quibusdam eaque non quis animi praesentium qui

unde similiq̃ue suscipit,